EJERCICIO 1

1. Calcula la complejidad ciclomática.
El **if** tiene 2 condiciones asi que 2+1= 3

2. Diseña el conjunto mínimo de pruebas para cubrir todas las ramas.
Un valor negativo, uno positivo y un cero

3. Implementa la clase SignoTest con JUnit.

```
@Test
void testSignoPositivo() { assertEquals( expected: 1, Main.signo( x: 5)); }

@Test
void testSignoNegativo() { assertEquals( expected: -1, Main.signo( x: -10)); }

@Test
void testSignoCero() { assertEquals( expected: 0, Main.signo( x: 0)); }
```

EJERCICIO 2

1. Calcula la complejidad ciclomática.
Hay 6 opciones, asi que 6+1 = 7

2. Diseña el conjunto mínimo de pruebas para cubrir todas las ramas.
edad negativa
edad < 0
edad < 6
edad < 12
edad < 18
edad < 25
edad < 60

3. Añade además casos límite especialmente relevantes.
0: El valor más bajo permitido
6: Niñez
12: Adolescencia
18: Juventud
25: Adultez
60: Vejez

4. Implementa ClasificarEdadTest

```java
@Test
void testEdadNegativa() {
    assertThrows(IllegalArgumentException.class, () -> {
        Main.clasificarEdad(-1);
    });
}
@Test
void testLimiteInferiorInfancia() {
    assertEquals( expected: "Infancia", Main.clasificarEdad(0));
}
@Test
void testTransicionNiñez() {
    assertEquals( expected: "Infancia", Main.clasificarEdad(5));
    assertEquals( expected: "Niñez", Main.clasificarEdad(6));
}
@Test
void testTransicionAdolescencia() {
    assertEquals( expected: "Niñez", Main.clasificarEdad(11));
    assertEquals( expected: "Adolescencia", Main.clasificarEdad(12)
}
@Test
void testTransicionJuventud() {
    assertEquals( expected: "Adolescencia", Main.clasificarEdad(17)
    assertEquals( expected: "Juventud", Main.clasificarEdad(18));
}
@Test
void testTransicionAdultez() {
    assertEquals( expected: "Juventud", Main.clasificarEdad(24));
    assertEquals( expected: "Adultez", Main.clasificarEdad(25));
}
@Test
void testTransicionVejez() {
    assertEquals( expected: "Adultez", Main.clasificarEdad(59));
    assertEquals( expected: "Vejez", Main.clasificarEdad(60));
}
@Test
void testVejezExtrema() {
    assertEquals( expected: "Vejez", Main.clasificarEdad(100));
}
```

EJERCICIO 3

1. Calcula la complejidad ciclomática.
El **for** un camino, el **if** otro camino asi que 2+1= 3

2. Diseña pruebas para cubrir:
    - array vacío,
    - array con todos negativos,
    - array con mezcla de positivos, negativos y ceros.

ARRAY VACÍO:
Entrada: []
Resultado: 0

ARRAY NEGATIVOS:
Entrada: [-1, -5]
Resultado: 0

ARRAY POSITIVOS, NEGATIVOS Y CEROS:
Entrada: [1, 0, -2]
Resultado: 1

3. Justifica qué ramas cubre cada prueba.

ARRAY VACÍO:
(El “i < datos.length” lo corta antes de entrar al bucle)

ARRAY NEGATIVOS:
(Entra al bucle pero no suma el contador porque son menor que 0)

ARRAY POSITIVOS, NEGATIVOS Y CEROS:
(Entra al bucle pero solo suma uno, “1” porque es el único mayor que 0)

4. Implementa ContarPositivosTest.

```
@Test
void testArrayVacio() {
    int[] datos = {};
    assertEquals( expected: 0, Main.contarPositivos(datos));
}

@Test
void testTodosNegativos() {
    int[] datos = {-1, -5};
    assertEquals( expected: 0, Main.contarPositivos(datos));
}

@Test
void testMezclaPositivosNegativosYCeros() {
    int[] datos = {1, 0, -2};
    // Solo el '1' es positivo (> 0), el resultado debe ser 1
    assertEquals( expected: 1, Main.contarPositivos(datos));
}
```

EJERCICIO 4

1. Calcula la complejidad ciclomática

Tiene 6 casos asi que 6+1= 7

El **if** tiene dos caso y el **case** tiene 4

2. Diseña los casos mínimos para cubrir todas las alternativas del switch y la validación inicial.

LÍMITE INFERIOR
Entrada: -1
Resultado: IllegalArgumentException

LÍMITE SUPERIOR
Entrada: 11
Resultado: IllegalArgumentException

PRIMER CASO (SUSPENSO)
Entrada: 3
Resultado: "SUSPENSO"

SEGUNDO CASO (SUFICIENTE)
Entrada: 5
Resultado: "SUFICIENTE"

TERCER CASO (BIEN)
Entrada: 6
Resultado: "BIEN"

CUARTO CASO (NOTABLE)
Entrada: 8
Resultado: "NOTABLE"

DEFAULT (SOBRESALIENTE)
Entrada: 10
Resultado: "SOBRESALIENTE"

3. Implementa CalificacionTest.

```java
@Test
void testNotaNegativa() {
    assertThrows(IllegalArgumentException.class, () -> Main.calificacion( nota: -1));
}
@Test
void testNotaExcesiva() {
    assertThrows(IllegalArgumentException.class, () -> Main.calificacion( nota: 11));
}
@Test
void testSuspenso() {
    assertEquals( expected: "SUSPENSO", Main.calificacion( nota: 2));
}
@Test
void testSuficiente() {
    assertEquals( expected: "SUFICIENTE", Main.calificacion( nota: 5));
}
@Test
void testBien() {
    assertEquals( expected: "BIEN", Main.calificacion( nota: 6));
}
@Test
void testNotable() {
    assertEquals( expected: "NOTABLE", Main.calificacion( nota: 7));
}
@Test
void testSobresaliente() {
    assertEquals( expected: "SOBRESALIENTE", Main.calificacion( nota: 9));
}
```

4. Indica qué casos podrían agruparse con @ParameterizedTest.
Todos lo que validan una nota

EJERCICIO 5

1. Implementa el método.

```
public static boolean esBisiesto(int anyo) {  no usages
    if ((anyo % 4 == 0 && anyo % 100 != 0) || (anyo % 400 == 0)) {
        return true;
    } else {
        return false;
    }
}
```

2. Calcula su complejidad ciclomática a partir de tu código.
Tiene 3 decisiones + 1= 4

3. Diseña los casos de prueba necesarios para cubrir todas las decisiones.

DIVISIBLE POR 4 Y NO POR 100
Entrada: 2024
Resultado: true

DIVISIBLE POR 100 Y NO POR 400
Entrada: 1900
Resultado: false

DIVISIBLE POR 400
Entrada: 2000
Resultado: true

NO DIVISIBLE POR 4
Entrada: 2023
Resultado: false

4. Implementa EsBisiestoTest.

```
@Test
void testNoDivisiblePorCuatro() {
    // Caso: No divisible por 4
    assertFalse(Main.esBisiesto( anyo: 2023));
}

@Test
void testBisiestoEstandar() {
    // Caso: Divisible por 4 y no por 100
    assertTrue(Main.esBisiesto( anyo: 2024));
}

@Test
void testSigloNoBisiesto() {
    // Caso: Divisible por 100 y no por 400
    assertFalse(Main.esBisiesto( anyo: 1900));
}

@Test
void testSigloBisiesto() {
    // Caso: Divisible por 400
    assertTrue(Main.esBisiesto( anyo: 2000));
}
```

EJERCICIO 6

1. Implementar el método.

```java
public static String jugar(String j1, String j2) {  no usages
    if (!esValida(j1) || !esValida(j2)) {
        throw new IllegalArgumentException("Jugada no válida");
    }

    if (j1.equals(j2)) {
        return "EMPATE";
    }

    if ((j1.equals("PIEDRA") && j2.equals("TIJERA")) ||
            (j1.equals("TIJERA") && j2.equals("PAPEL")) ||
            (j1.equals("PAPEL") && j2.equals("PIEDRA"))) {
        return "GANA JUGADOR 1";
    } else {
        return "GANA JUGADOR 2";
    }
}

private static boolean esValida(String jugada) {  2 usages
    return jugada.equals("PIEDRA") || jugada.equals("PAPEL") || jugada.equals("TIJERA");
}
```

2. Calcular la complejidad ciclomática.
3 casos de los **if** + 1 = 4

3. Diseñar los tests mínimos para cubrir todas las ramas.
JUGADA INVÁLIDA:
Entrada: j1="PIEDRA", j2="ARCO"
Resultado: IllegalArgumentException

EMPATE:
Entrada: j1="TIJERA", j2="TIJERA"
Resultado: "EMPATE"

VICTORIA JUGADOR 1:
Entrada: j1="PIEDRA", j2="TIJERA"
Resultado: "GANA JUGADOR 1"

VICTORIA JUGADOR 2:
Entrada: j1="TIJERA", j2="PIEDRA"
Resultado: "GANA JUGADOR 2"

4. Implementar PiedraPapelTijeraTest.

```java
@Test
void testJugadaInvalida() {
    assertThrows(IllegalArgumentException.class, () -> {
        Main.jugar("PIEDRA", "ARCO");
    });
}

@Test
void testEmpate() {
    assertEquals( expected: "EMPATE", Main.jugar("PAPEL", "PAPEL"));
}

@Test
void testGanaJugador1() {
    // Probamos una de las condiciones (Piedra gana a Tijera)
    assertEquals( expected: "GANA JUGADOR 1", Main.jugar("PIEDRA", "TIJERA"));
}

@Test
void testGanaJugador2() {
    // Probamos que si J1 pierde, el else devuelve correctamente J2
    assertEquals( expected: "GANA JUGADOR 2", Main.jugar("TIJERA", "PIEDRA"));
}
```

EJERCICIO 7

1. Implementar el método.

```java
public static String evaluarPassword(String password) {  no usages
    if (password == null) {
        return "DEBIL";
    }

    int puntos = 0;
    if (password.length() >= 8){
        puntos++;
    }
    if (password.matches( regex: ".*[A-Z].*")) {
        puntos++;
    }
    if (password.matches( regex: ".*[a-z].*")) {
        puntos++;
    }
    if (password.matches( regex: ".*[0-9].*")) {
        puntos++;
    }
    if (password.matches( regex: ".*[^a-zA-Z0-9].*")) {
        puntos++;
    }

    if (puntos >= 0 && puntos <= 2) {
        return "DEBIL";
    }
    if (puntos >= 3 && puntos <= 4) {
        return "MEDIA";
    }
    return "FUERTE";
}
```

2. Calcular complejidad ciclomática.

8 casos de los **if** + 1 = 9

3. Diseñar casos de prueba representativos.

MUY DÉBIL:
Entrada: password="abc"
Resultado: "DEBIL" (1 punto)

LÍMITE DÉBIL:
Entrada: password="abc123"
Resultado: "DEBIL" (2 puntos)

MEDIA (3):
Entrada: password="Abc123"
Resultado: "MEDIA" (4 puntos)

MEDIA (4):
Entrada: password="Abc12345"
Resultado: "MEDIA" (4 puntos)

FUERTE:
Entrada: password="Abc12345!"
Resultado: "FUERTE" (5 puntos)

4. Implementar PasswordTest.

```java
@Test
void testPasswordDebil() {
    assertEquals( expected: "DEBIL", Main.evaluarPassword("abc"));
    assertEquals( expected: "DEBIL", Main.evaluarPassword("abc12"));
}

@Test
void testPasswordMedia() {
    assertEquals( expected: "MEDIA", Main.evaluarPassword("Abc1234"));
}

@Test
void testPasswordFuerte() {
    assertEquals( expected: "FUERTE", Main.evaluarPassword("Abc12345!"));
}

@Test
void testNulo() {
    assertEquals( expected: "DEBIL", Main.evaluarPassword(null));
}
```